

الفصل السادس (Section 6)

1.6- الدوال (Functions):-

ورثت اللغة C++ من اللغة C مكتبة ضخمة و غنية بدوال تقوم بتنفيذ العمليات الرياضية، التعامل مع السلاسل والأحرف، الإدخال والإخراج، اكتشاف الأخطاء والعديد من العمليات الأخرى المفيدة مما يسهل مهمة المبرمج الذي يجد في هذه الدوال معيناً كبيراً له في عملية البرمجة. يمكن للمبرمج كتابة دوال تقوم بأداء عمليات يحتاج لها في برامجه وتسمى مثل هذه الدوال انها دوال معرفة من قبل المبرمج (Programmer-defined functions). تعرف الدالة على أنها جملة أو مجموعة جمل أو تعليمات ، ذات كيان خاص، تقوم بعملية أو مجموعة عمليات ، سواء عمليات إدخال أو إخراج أو عمليات حسابية أو منطقية ، وتحتل الدالة موقعا من البرنامج ، أي أنها جزء منه ، أو يمكن القول أن لغة C++ تتكون من مجموعة من الدوال.

كما تعرف الدالة عبارة عن مجموعة من العبارات المصممة لإنجاز مهمة . لقد أظهرت التجربة أن أفضل طريقة لتطوير وصيانة برنامج كبير هي بنائه من قطع أصغر (وحدات نمطية). تسمى الوحدات النمطية في C++ بالدوال. الدوال مفيدة جدا لقراءة وكتابة وتصحيح وتعديل البرامج المعقدة، يمكن أيضاً دمجها بسهولة في البرنامج الرئيسي. في C++، main () نفسها هي دالة تعني أن الوظيفة الرئيسية هي استدعاء الوظائف الأخرى لأداء مهام مختلفة. الدوال تمكن المبرمج من تقسيم البرنامج إلى وحدات modules، كل دالة في البرنامج تمثل وحدة قائمة بذاتها، ولذا نجد أن المتغيرات المعرفة في الدالة تكون متغيرات محلية (Local) ونعني بذلك أن المتغيرات تكون معروفة فقط داخل الدالة. أغلب الدوال تمتلك لائحة من الوسائط (Parameters) والتي هي أيضاً متغيرات محلية.

المزايا الرئيسية لاستخدام الدوال هي:-

- 1 - تساعد الدوال المخزنة في ذاكرة الحاسب على اختصار البرنامج إذ يكتفى باستعادتها باسمها فقط لتقوم بالعمل المطلوب.
- 2 - تساعد البرامج المخزنة في ذاكرة الحاسب ، أو التي يكتبها المبرمج على تلافي عمليات التكرار في خطوات البرنامج التي تتطلب عملاً طويلاً وشاقاً.
- 3 - تساعد الدوال الجاهزة على تسهيل عملية البرمجة نفسها.
- 4 - توفر مساحة من الذاكرة المطلوبة.
- 5 - اختصار عمليات زمن البرمجة وتنفيذ البرنامج بأسرع وقت ممكن.

وللتدليل على أهمية الدوال في برمجة C++

فمثلاً لو أردنا كتابة خوارزمية لخطوات صنع كأس من الشاي فأننا نكتب ما يأتي:

١ -ضع الماء في غلاية الشاي.

٢ -سخن الماء حتى يغلي.

٣ -أضف شاياً إلى الماء.

٤ - أطفئ النار.

٥ - أصب شاياً في كأس.

٦ - أضف سكرًا إليه.

افرض الآن أننا نود طلب كأس من الشاي من مقهى مجاور : أن خطوات الخوارزمية التي نحتاجها الآن هي خطوه واحده فقط وهي:

١ -استدع كأس من الشاي.

تخيل الآن كم وفرنا من الخطوات لو استعملنا الدوال الجاهزة (أو التي يجهزها المبرمج من قبل) بدلا من خطواتها التفصيلية وبخاصة في برنامج يتطلب حسابات وعمليات كثيرة وكم يكون البرنامج سهلاً وواضحاً وقتذاك.

2.6- تحديد أو تعريف الدالة (Defining a Function):-

أن الدالة قد تعتمد على متغير أو أكثر ، وقد لا تعتمد على أي متغير ، وفي كلا الحالتين ، يستعمل بعد اسم الدالة قوسين () سواء كان بينهما متغيرات أم لا .

يأخذ تعريف الدوال في C++ الشكل العام التالي:

return-value-type function-name (parameter list)

```
{  
    declarations and statements  
}
```

حيث:

return-value-type: نوع القيمة المعادة بواسطة الدالة والذي يمكن أن يكون أي نوع من أنواع بيانات C++. وإذا كانت الدالة لا ترجع أي قيمة يكون نوع إعادتها **void**.

function-name: اسم الدالة والذي يتبع في تسميته قواعد تسمية المعرفات (identifiers).

parameter list: هي لائحة الوسيطات الممررة إلى الدالة وهي يمكن أن تكون خالية (void) أو تحتوى على وسيطة واحدة أو عدة وسائط تفصل بينها فاصلة ويجب ذكر كل وسيطة على حدة.

void square(int a, int b) → a,b are the formal arguments.

float display(void) → function without formal arguments

declarations and statements: تمثل جسم الدالة والذي يطلق عليه في بعض الأحيان block. يمكن أن يحتوى block على إعلانات المتغيرات ولكن تحت أي ظرف لا يمكن أن يتم تعريف دالة داخل جسم دالة أخرى. السطر الأول في تعريف الدالة يدعى المصريح declarator والذي يحدد اسم الدالة ونوع البيانات التي تعيدها الدالة وأسماء وأنواع وسيطاتها.

3.6- استدعاء الدالة (Function Call):-

يتم استدعاء الدالة (التسبب بتنفيذها من جزء آخر من البرنامج، العبارة التي تفعل ذلك هي استدعاء الدالة) يؤدي استدعاء الدالة إلى انتقال التنفيذ إلى بداية الدالة.

يمكن تمرير بعض الوسيطات إلى الدالة عند استدعائها وبعد تنفيذ الدالة يعود التنفيذ للعبارة التي تلي استدعاء الدالة. بإمكان الدالة أن تعيد قيم إلى العبارة التي استدعتها. ويجب أن يسبق اسم الدالة في معرفتها وإذا كانت الدالة لا تعيد شيئاً يجب استعمال الكلمة الأساسية void كنوع إعادة لها للإشارة إلى ذلك.

هنالك ثلاث طرق يمكن بها إرجاع التحكم إلى النقطة التي تم فيها استدعاء الدالة:

- إذا كانت الدالة لا ترجع قيمة يرجع التحكم تلقائياً عند الوصول إلى نهاية الدالة.
- باستخدام العبارة **return;**
- إذا كانت الدالة ترجع قيمة فالعبارة **return expression;** تقوم بإرجاع قيمة التعبير expression إلى النقطة التي استدعتها.

Example 1:- Write C++ program, to uses a function called square to calculate the number squares from 1 to 10?

```
#include<iostream.h>  
int square(int y)  
{  
    return y*y;  
}  
main()  
{  
for(int x=1;x<=10;x++)  
    cout<<square(x)<<" ";  
    cout<<endl;  
}
```


يتم استدعاء الدالة square داخل الدالة main وذلك بكتابة square(x). تقوم الدالة square بنسخ قيمة x في الوسيط y ثم تقوم بحساب y*y ويتم إرجاع النتيجة إلى الدالة main مكان استدعاء الدالة square ، حيث يتم عرض النتيجة وتكرر هذه العملية عشر مرات باستخدام حلقة التكرار for. تعريف الدالة square () يدل على أنها تتوقع وسيطة من النوع int . و int التي تسبق اسم الدالة تدل على أن القيمة المعادة من الدالة square هي من النوع int أيضاً . العبارة return تقوم بإرجاع ناتج الدالة إلى الدالة main.

Example 2:- Write C++ program, to print the statement “Mustansiriyah of University” by using function ?

```
#include<iostream.h>
void printmessage ( )
{
cout << “University of Technology”;
}
void main ( )
{
printmessage( );
}
```

Example 3:- Write C++ program, to prints the largest number between two numbers entered by the user using function?

```
#include <iostream.h>
int x,y;
max( )
{
if (x>y)
cout<<x;
else
cout<<y;
}
main()
{
cin>>x>>y;
max( );
return0;
}
```

Example 4:- Write C++ program using function to calculate the average of two numbers entered by the user in the main program?

```
#include<iostream.h>
float aver (int x1, int x2)
{
float z;
z = ( x1 + x2) / 2.0;
return ( z);
}
void main( )
```

```

{
float x;
int num1,num2;
cout << "Enter 2 positive number \n";
cin >> num1 >> num2;
x = aver (num1, num2);
cout << x;
}

```

Example 5:- Write C++ program, using function, to find the summation of the following Series?

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + 3^2 + \dots + n^2$$

```

#include<iostream.h>
int summation ( int x)
{
int i = 1, sum = 0;
while ( i <= x )
{
sum += i * i ;
i++;
}
return (sum);
}
void main ( )
{
int n ,s;
cout << "enter positive number";
cin >> n;
s = summation ( n );
cout << "sum is: " << s << endl;
}

```

Example 6:- Write a function to find the largest integer among three integers entered by the user

in the main function?

```

#include <iostream.h>
int max(int y1, int y2, int y3)
{
int big;
big=y1;
if (y2>big) big=y2;
if (y3>big) big=y3;
return (big);
}
void main( )
{
int largest,x1,x2,x3;

```



```

cout<<"Enter 3 integer numbers:";
cin>>x1>>x2>>x3;
largest=max(x1,x2,x3);
cout<<largest;
}

```

Example 7:-Write program in C++, using function, presentation for logic gates (AND, OR ,NAND, X-OR,NOT) by in A,B enter from user?

```

#include<iostream.h>
void ANDF(int a,int b)
{
cout<<(a&&b);
}
void ORF(int a,int b)
{
cout<<(a||b);
}
void XORF(int a,int b)
{
cout<<(a^b);
}
void NOTF(int a)
{
cout<<(!a);
}
void main()
{ char s;int a,b;
cout<<"Enter the value A,B :";
cin>>a>>b;
cout<<"Enter the select value \n";
cout<<"\ta--(AND gate)\n\to--(OR gate)\n\tx--(X-OR)\n\tn--(NOT
gate)\n\te--(<<EXIT>> :";
cin>>s;
switch(s)
{
case 'a':ANDF(a,b);break;
case 'o':ORF(a,b);break;
case 'x':XORF(a,b);break;
case 'n':NOTF(a);cout<<" ";NOTF(b);break;
case 'e':break;
default:cout<<":bad choose";
}
}

```

Example 8:-write C++ program, using function, to find (search) X value in array, and return the index of it's location?

```

#include<iostream.h>
int search( int a[ ], int y)

```

```

{
int i= 0;
while ( a [ i ] != y )
i++;
return ( i );
}
void main ( )
{
int X, f;
int a [ 10 ] = { 18, 25, 36, 44, 12, 60, 75, 89, 10, 50 };
cout << "enter value to find it: ";
cin >> X;
f= search (a, X);
cout << "the value " << X << " is found in location " << f;
}

```

4.6- نموذج الدالة (Function Prototype):-

عندما نحتاج لاستدعاء دالة ما فإنه يحتاج إلى معرفة اسم الدالة وعدد وسيطاتها وأنواعها ونوع قيمة الإعادة، لذا علينا كتابة نموذج أو (تصريح) للدالة قبل إجراء أي استدعاء لها وتصريح الدالة هو سطر واحد عبارة عن اسم الدالة وعدد وسيطاتها وأنواعها ونوع القيمة المعادة بواسطة الدالة. يشبه تصريح الدالة، السطر الأول في تعريف الدالة، لكن تليه فاصلة منقوطة. فمثلا في تصريح الدالة التالي:-

int anyfunc(int);

النوع int بين القوسين يخبر بأن الوسيط الذي سيتم تمريره إلى الدالة سيكون من النوع int و int التي تسبق اسم الدالة تشير إلى نوع القيمة المعادة بواسطة الدالة. عند تعريف الدالة هناك طريقتين الأولى تدعى definition وهي انه الدالة تكون مكتوبة قبل الاستدعاء اي يكون الاستدعاء من الاسفل الى الاعلى كما هو مكتوب في كل الامثلة السابقة في الدوال. اما الطريقة الثانية تدعى declaration كما تدعى احيانا header وهي انه الدالة تكون مكتوبة بعد الاستدعاء اي يكون الاستدعاء من الاعلى الى الاسفل كما في المثال التالي.

Example:- Write C++ program, using function, to print the cub of values?

```
#include<iostream.h>
```

```
int cube(int);
```

```
void main()
```

```
{
```

```
cube(2);
```

```
}
```

```
int cube(int x)
```

```
{
```

```
return x*x*x;
```

```
}
```

كما ان استدعاء الدالة ليس شرطاً ان يكون من دالة main فقط، اذ يمكن ان تستدعي من دالة أخرى وتلك الدالة تستدعي من main كما هو موضح في المثال التالي :-

```
#include<iostream.h>
```

```
Int cube(int);
```

```
void call()
```

```
{
```

```
cout<<cube(2);
```



```

}
void main()
{
call();
}
int cube(int x)
{
return x*x*x;
}

```

5.6- تمرير الوسيطات (Passing Parameters):-

الوسيطات او المعلمات بالاساس يمكن تصنيفها الى مجموعتين (الفعلية والرسمية).
الوسيطه الفعلية(actual arguments): هي متغير أو تعبير مضمن في استدعاء دالة الذي يحل محل المعلمة الرسمية التي هو جزء من إعلان الدالة. في بعض الأحيان ، قد تكون دالة استدعت جزء من البرنامج مع بعض المعلمات وهذه تُعرف بالمعلمة الفعلية.

الوسائط الرسمية(formal arguments): هي المعلمات الموجودة في تعريف الدالة والتي يمكن أيضا أن تسمى بالمعلمة الوهمية أو محدودة المتغيرات. عند استدعاء الدالة ، تكون المعلمات الرسمية استبدلت من قبل المعلمات الفعلية. قد يتم الإعلان عن الوسائط الرسمية بنفس الاسم أو بأسماء مختلفة عند استدعاء جزء من البرنامج أو في دالة الاستدعاء ولكن يجب أن تكون أنواع البيانات هي نفسها في كلا المجموعتين.

example:

```

#include <iostream.h>
Void main()
{
Int x,y;
Void output (int x, int y); // function declaration
—
—
Output (x,y); // x and y are actual arguments
}
Void output (int a, int b) // formal arguments
{
// body of function
}

```

هناك طريقتان رئيسيتان لتمرير المعلمات أو الوسيطات إلى البرنامج :-

1- التمرير بالقيمة (passing by value)

2- التمرير بالمرجع (passing by reference)

التمرير بالقيمة (passing by value):-

عندما يتم تمرير المعلمات حسب القيمة ، نسخة من المعلمات يتم أخذ القيمة من دالة الاستدعاء وتمريرها إلى الدالة المطلوبة. لن تتغير المتغيرات الأصلية داخل دالة الاستدعاء ، بغض النظر عن التغييرات التي تجريها الدالة عليها. لنفرض أننا لدينا متغيرين صحيحين في برنامج ونريد استدعاء دالة تقوم بتبديل قيمتي الرقمين ، لنفرض أننا عرفنا الرقمين كالآتي:

```

int x=1;
int y=2;

```

تري هل تقوم الدالة التالية بتبديل القيمتين

```
#include <iostream.h>
void swap (int a, int b)
{
    int temp =a;
    a=b;
    b=temp;
}
main ( )
{
    int x= 1;
    int y= 2;
    swap (x, y);
}
```

تقوم هذه الدالة بتبديل قيمتي a و b ، لكن إذا استدعينا هذه الدالة كالآتي:

```
swap( x,y);
```

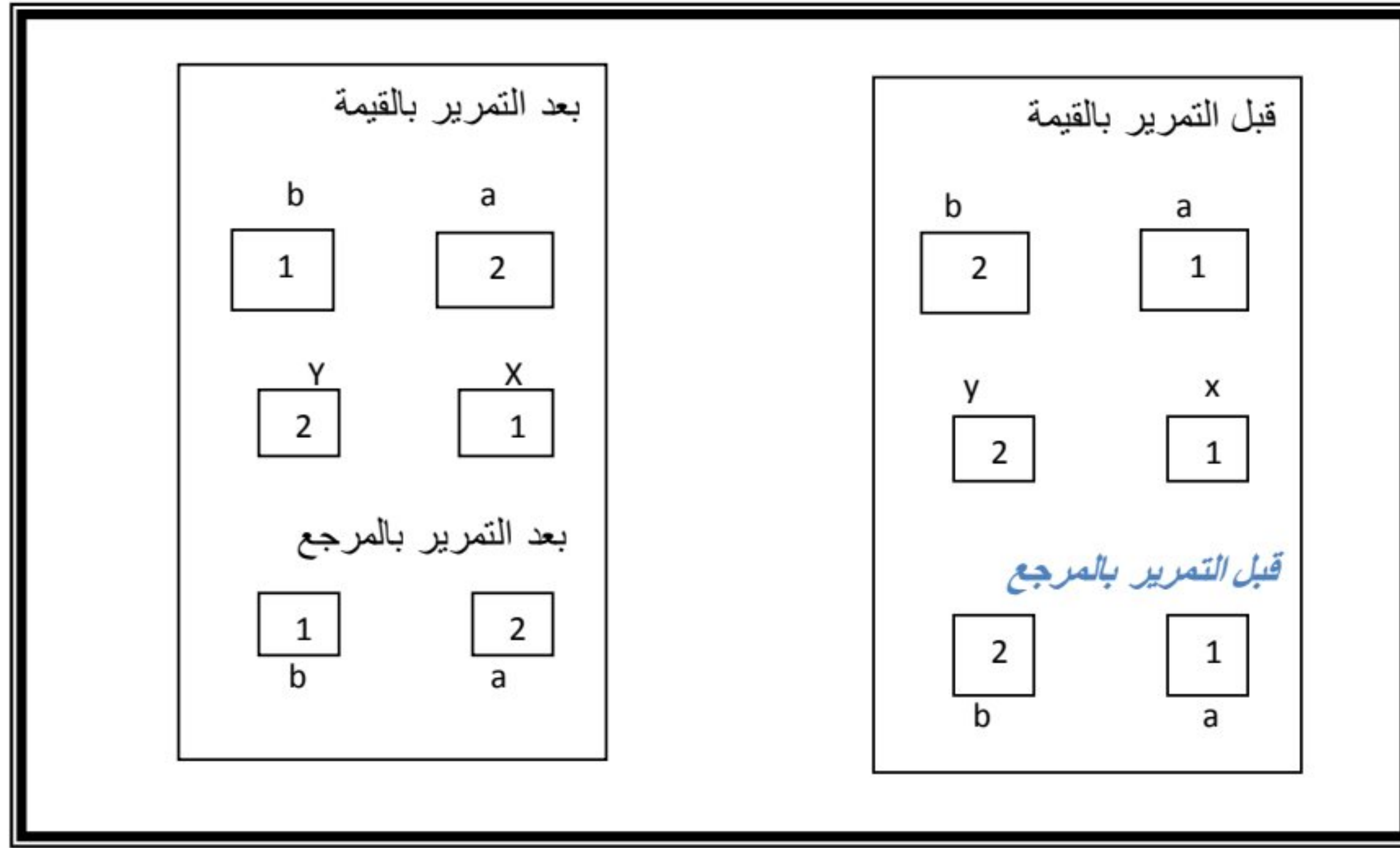
سنجد أن قيمتي x و y لم تتغير وذلك لأن الوسيطات الاعتيادية للدالة يتم تمريرها بالقيمة وتنشئ الدالة متغيرات جديدة كلياً هي a و b في هذا المثال لتخزين القيم الممررة إليها وهي (1,2) ثم تعمل على تلك المتغيرات الجديدة وعليه عندما تنتهي الدالة ورغم أنها قامت بتغيير a إلى 2 و b إلى 1 لكن المتغيرات x و y في استدعاء الدالة لم تتغير.

التمرير بالمرجع (passing by reference):-

عندما يتم تمرير المعلومات بالرجوع يتم نسخ عناوينهم إلى الوسيطات المقابلة في الدالة المدعومة ، بدلاً من نسخ قيمها. وبالتالي ، عادةً ما تستخدم المؤشرات في قائمة وسيطات الدوال لتلقي المراجع التي تم تمريرها. التمرير بالمرجع هو طريقة تمكن الدالة swap () من الوصول إلى المتغيرات الأصلية x و y والتعامل معها بدلاً من إنشاء متغيرات جديدة. ولإجبار تمرير الوسيطة بالمرجع نضيف الحرف & إلى نوع بيانات الوسيطة في تعريف الدالة وتصريح الدالة .

```
#include <iostream.h>
void swap (int& a, int & b)
{
    cout <<"Original value of a is " << a<<endl;
    int temp =a;
    a=b;
    b=temp;
    cout <<"swapped value of a is " << a<<endl;
}
main ( )
{
    int x= 1;
    int y= 2;
    swap (x, y);
    return 0;
}
```

الحرف & يلي int في التصريح والتعريف وهو يبلغ المصرف أن يمرر هذه الوسيطات بالمرجع، أي أن الوسيطة a هي مرجع إلى x و b هي مرجع إلى y ولا يستعمل & في استدعاء الدالة.



شكل يوضح طريقتي التمرير بالمرجع والتمرير

طريقة التمرير بالمرجع أكثر فاعلية وتوفر سرعة تنفيذ أعلى من طريقة التمرير بالقيمة ، ولكن التمرير بالقيمة أكثر مباشرة وسهلة الاستخدام.

5.6- أنواع الدوال (Types of Functions):-

يمكن تصنيف الدوال المعرفة من قبل المستخدم بالطرق الثلاث التالية بناءً على الوسائط الرسمية التي تم تمريرها واستخدام بيان الإرجاع ، وبناءً على ذلك ، هناك ثلاث دوال محددة من قبل المستخدم:

1. يتم استدعاء الدالة دون تمرير أي وسيطة رسمية من الجزء المتصل بالبرنامج وأيضًا لا تقوم الدالة بإرجاع أي قيمة إلى الدالة التي تم استدعاؤها.
2. يتم استدعاء الدالة باستخدام الوسائط الرسمية من جزء الاستدعاء في البرنامج ، لكن الدالة لا تُرجع أي قيمة إلى جزء الاستدعاء.
3. يتم استدعاء الدالة باستخدام الوسائط الرسمية من جزء الاستدعاء في البرنامج الذي يعيد قيمة إلى بيئة الاستدعاء.

فيما يلي برنامجان يقومان بطباعة مربع العدد باستخدام عبارة الإرجاع مرة وتارة عدم استخدامها.

```
# include <iostream.h>
void square(int n)
{
float value;
value=n*n;
cout<<"i="<<n<<"square="<<value<<endl;
}
void main()
{
int max;
cout<<"Enter a value for n ?\n";
cin>>max;
for (int i=0;i<=max-1;++i)
square (i)
}
```

```
# include <iostream.h>
float square(float n)
{
float value;
value=n*n;
return (value);
}
void main()
{
float l,max,value;
max=1.5;
i=-1.5;
while (i<=max) {
value=square(i);
cout<<"i="<<i<<"square="<<value<<endl;
i=i+0.5;
}
}
```

Example:- Write program in C++, using function print sum for two integer values?

```
#include<iostream.h>
void sum()
{
int x = 5 , y = 4;
int z = x + y;
cout<<z;
}
void main()
{
sum();
}
```

لاحظ هنا الدالة من نوع void اي لا ترجع شيء و بنفس الوقت لا تمرر وسيطات او معلومات اي الاقواس فارغة بالتالي لا تأخذ شيء.

6.6- دالة اعادة الاستدعاء (Recursive Functions):-

دالة التي تدعو نفسها مباشرة أو غير مباشرة مرارا وتكرارا هي المعروف بأسم دالة اعادة الاستدعاء. و هذه الدوال مفيدة جدًا أثناء إنشاء هياكل البيانات مثل القوائم المرتبطة (linked lists) والقوائم المرتبطة المزدوجة (double linked lists) والأشجار (trees). هناك اختلاف واضح بينها وبين الدوال العادية. سيتم استدعاء الدالة العادية بواسطة الدالة الرئيسية (main) كلما تم استخدام اسم الدالة، في حين سيتم استدعاء دالة اعادة الاستدعاء من تلقاء نفسها بشكل مباشر أو غير مباشر طالما تم استيفاء الشرط المحدد، فهي تستخدم عوضا عن ايعازات التكرار (loop statements) و التي تتطلب نهاية لحقة التكرار (end point) لذا نستخدم مع هذا النوع من الدوال (Recursive Functions) عبارات التحكم (control statement)، لكن البرنامج المستخدم فيه دالة اعادة

الاستدعاء تتطلب ذاكرة اكبر من البرنامج المستخدم فيه ايعازات التكرار و بالتالي عملية تنفيذه تكون ابطأ و رغم ذلك فهو افضل في البرامج المعقدة ، و دالة اعادة الاستدعاء تعمل وفق مبدأ المكس (stack).
المثال ادناه يوضح عمل دالة اعادة الاستدعاء :-

```
#include <iostream.h>
void main(void)
{
void func1(); //function declaration
_____
_____
func1(); //function calling
}
void func1() //function definition
{
_____
_____
func1(); //function calls recursively
}
```

Example 1:-write program to find the sum of the given non negative integer numbers using a recursive function $\text{sum}=1+2+3+4+\dots+n$

```
#include <iostream.h>
void main(void)
{
int sum(int);
int n,temp;
cout<<"Enter any integer number"<<endl;
cin>>n;
temp=sum(n);
cout<<"value="<<n<<"and its sum="<<temp;
}
int sum(int n) //recursive function
{
int sum(int); //local function declaration
int value=0;
if (n==0)
return (value);
else
value=n+sum(n-1);
return (value);
}
```


Example 2:- write program to find the factorial ($n!$) of the given number using the recursive function. Its the product of all integers from 1 to n (n is non negative)
(so $n!=1$ if $n=0$ and $n!=n(n-1)$ if $n>0$)

```
#include <iostream.h>
void main(void)
{
long int fact (long int);
int x,n;
cout<<"Enter any integer number"<<endl;
cin>>n;
x=fact(n);
cout<<"value="<<n<<"and its factorial=";
cout<<x<<endl;
}
long int fact (long int n) //recursive function
{
long int fact(long int); //local function declaration
int value =1;
if (n==1)
return(value)
else
{
value=n*fact(n-1);
return(value);
}
}
```

Home Work (6)

- Q1: Write a C++ program, using function, to counts uppercase letter in a 20 letters entered by the user in the main program?
- Q2: Write a C++ program, using function, that reads two integers (feet and inches) representing distance, then converts this distance to meter?
Note: 1 foot = 12 inch
1 inch = 2.54 Cm
- Q3: Write a C++ program, using function, which reads an integer value (T) representing time in seconds, and converts it to equivalent hours (hr), minutes (mn), and seconds (sec), in the following form:- **hr : mn : sec**
- Q4: Write a C++ program, using function, to see if a number is an integer (odd or even) or not an integer?
- Q5: Write a C++ program, using function, to represent the permutation of n?
- Q6: Write a C++ program, using function, to inputs a student's average and returns 4 if student's average is 90-100, 3 if the average is 80-89, 2 if the average is 70-79, 1 if the average is 60-69, and 0 if the average is lower than 60?
- Q7: The Fibonacci Series is: 0, 1, 1, 2, 3, 5, 8, 13, 21, ... It begins with the terms 0 and 1 and has the property that each succeeding term is the sum of the two preceding terms, Write a C++ program, using function, to calculate the nth Fibonacci number?
- Q8: Write a C++ program, using function, to calculate the factorial of an integer entered by the user at the main program?
- Q9: Write a C++ program, using function, to evaluate the following equation:
$$z = \frac{x! - y!}{(x - y)!}$$
- Q10: Write a C++ program, using function, to test the year if it's a leap or not.
Note: use $y \% 4 == 0 \ \&\& \ y \% 100 != 0 :: y \% 400 == 0$
- Q11: Write a C++ program, using function, to find x^y .
Note: use pow instruction with math.h library.
- Q12: Write C++ program, using function, to inverse an integer number: For example:
765432 $\longrightarrow$ 234567
- Q13: Write C++ program, using function, to find the summation of student's marks, and it's average, assume the student have 8 marks?

Q14: Write C++ program, using function, to convert any char. From capital to small or from small to capital?

Q15: Write C++ program using recursive function to find the power of numbers?

Q16: Write a C++ program, using function, to find if the array's elements are in order or Not?

Q17: Write a C++ program, using function, to compute the number of zeros in the array?

Q18: Write a C++ program, using function, to find the value of array C from add array A and array B. $C[i] = A[i] + B[i]$;

Q19: Write a C++ program, using function, to multiply the array elements by 2?

$$A[i] = A[i] * 2;$$

Q20: Write a C++ program, using function, to reads temperatures over the 30 days and calculate the average of them?

Q21: Write a C++ program, using function, to merge two arrays in one array?